Cullinos QA — Quick Start Guide

For new testers. Read this first, then use the detailed Manual Test Plan and Test Run Sheet.

What is Cullinos?

Cullinos is Rkyves's restaurant platform. One backend powers several **web apps**, each for a different job:

Who uses it	App	What they do
Rkyves platform team	**Super Admin**	Onboard restaurants, manage subscriptions
Restaurant owner / manager	**Admin**	Menu, staff, orders, reports, settings
Multi-outlet chains	**Management**	Compare outlets, stock transfers, franchise
Floor staff	**Waiter**	Take table orders or start QR sessions for guests
Cashier	**POS**	Counter / takeaway billing — https://pos.cullinos.com
Kitchen	**KDS**	Kitchen display — https://kds.cullinos.com
Guest / customer	**Cullinos App**	Order online via QR or link (guest OK; optional login)
Public	**Marketing site**	cullinos.com — no login

Your test setup: Production environment, **fresh restaurant** created via Super Admin, **one Waiter** and **one Cashier** for floor/POS tests.

Also read **WHATS_NEW.md** for features added in this cycle.

Full click-by-click guide: COMPLETE_FEATURE_TEST_GUIDE.md — every portal, page, and major action.

Portal URLs (Production)

Bookmark these. All production testing uses these domains.

Main apps (login required)

App	URL	Who logs in
Super Admin	https://platform.cullinos.com	Platform ops (Rkyves)
Admin (owner dashboard)	https://admin.cullinos.com	Restaurant owner
Management (chains)	https://manage.cullinos.com	Owner (Enterprise plan)
Cullinos Waiter (floor)	Android app (Play Store)	Waiter staff
POS (cashier)	https://pos.cullinos.com	Cashier staff
KDS (kitchen)	https://kds.cullinos.com	Kitchen staff
Admin Portal POS	https://admin.cullinos.com/pos	Owner / staff with POS access

No login

App	URL	Notes
Cullinos App	https://guest.cullinos.com/`{orgSlug}`/`{outletSlug}`	Replace slugs after tenant is created
QR session link	`…?session={token}`	Generated by waiter — expires when session ends
Marketing website	https://cullinos.com	Public pages

Example storefront (demo org — do not use for your QA run if testing fresh tenant):

`https://guest.cullinos.com/demo-restaurant/main-outlet`

API & developer tools

Tool	URL	Purpose
API health	https://api.cullinos.com/api/v1/health	Quick "is API up?" check
API DB health	https://api.cullinos.com/api/v1/health/db	Database connectivity
Swagger (API docs)	https://api.cullinos.com/docs	Test API-only features, create tables if needed

QR table ordering (waiter → guest)

1. Waiter opens **Cullinos Waiter** (Android) → tap a table → **Show QR to customers**
2. Guest scans QR (or opens copied link) → adds items → checkout
3. Items appear on the **same table order** in waiter app and on KDS
4. Waiter taps **End session** when the table is cleared (old QR stops working)

Pickup / Order Display setup:

1. Admin → **Displays** (`/displays`) — copy CDS / Pickup URL (legacy `/cds` and `/pickup-queue` redirect here).
2. Copy it and open in another tab, tablet, or TV — no login required for the public board.
3. Place a takeaway/counter order to see it appear in the queue.

Local dev (optional fallback)

For development only — production QA uses the URLs above:

App	Local URL
POS	http://localhost:5173
KDS	http://localhost:5174
KDS Order Display	http://localhost:5174/?outletId=YOUR_OUTLET_ID&mode=pickup

---

Credentials — fill this in before testing

> **Important:** Do **not** put real passwords in files you commit to git.
> Ask your team lead for production passwords, or use a password manager.
> Copy this section into `docs/qa/runs/RUN-YYYYMMDD/CREDENTIALS_LOG.md` and fill it there (that folder is gitignored).

Where passwords come from

Role	How you get login
Super Admin	Team lead shares platform login (stored locally in `secrets-export.txt` — **never commit that file**)
Owner	Created when Super Admin **onboards** a new restaurant — you choose email/password during onboard
Waiter (your 1 employee)	Owner creates in **Admin** → **Staff** — you set email/password there
Cashier	Same as Waiter — create in Admin → Staff for POS
Guest customer	No login required — use incognito; optional login modal is extra

Credential worksheet (copy & fill in)

```
RUN ID:          RUN-YYYYMMDD
Tester name:
Date:

── PLATFORM (Rkyves) ──────────────────────────
Super Admin URL:   https://platform.cullinos.com
Super Admin email: _____________________________
Password:          (password manager / ask team lead)

── QA RESTAURANT (created this run) ────────────
Restaurant name:   e.g. QA Test Kitchen
Org slug:          _____________________________  ← needed for storefront URL
Outlet name:       e.g. Main Outlet
Outlet slug:       _____________________________
Outlet ID:         _____________________________  ← needed for KDS URL
Plan:              enterprise

Owner URL:         https://admin.cullinos.com
Owner email:       _____________________________
Owner password:    (password manager)

Storefront URL:    https://guest.cullinos.com/{orgSlug}/{outletSlug}

── STAFF (1 employee) ──────────────────────────
Name:              _____________________________
Role:              Waiter
Waiter URL:        https://waiter.cullinos.com
Email:             _____________________________
Password:          (password manager)

── OPTIONAL ────────────────────────────────────
Cashier email:     _____________________________  (only if testing POS)
Management URL:    https://manage.cullinos.com  (login as owner)
```

Demo credentials (local dev only — NOT for production QA)

If you run the app **locally** with `npm run db:seed`, these demo accounts exist:

Role	Email	Password
Super Admin	`SEED_SUPER_ADMIN_EMAIL`	`SEED_SUPER_ADMIN_PASSWORD` (set before seeding)
Owner	`owner@cullinos.com`	`demo1234`

Local app ports: API `3000`, Admin `5181`, etc. — Waiter is Flutter `apps/waiter_mobile`; see root README.md.

Production QA uses a fresh tenant, not these demo accounts.

Who tests what?

One QA person switches roles; one employee tests Waiter only.

Role	Person	App	When
Super Admin	You	platform.cullinos.com	Day 1 — onboard restaurant
Owner	You	admin.cullinos.com	Day 1–4 — setup, admin, reports
Waiter	Your employee	Cullinos Waiter Android	Day 2 — table orders
Guest	You (incognito)	guest.cullinos.com/...	Day 2 — online checkout
Owner	You	manage.cullinos.com	Day 3 — chain features

You need **two browsers**: normal window (staff/owner) + **incognito** (guest customer).

Start testing in 30 minutes

Step 1 — Preflight (5 min)

Open in browser and confirm each works:

1. https://api.cullinos.com/api/v1/health → should show healthy JSON
2. https://api.cullinos.com/api/v1/health/db → database OK
3. https://platform.cullinos.com → Super Admin login page loads

Checklist: README preflight

Step 2 — Create QA restaurant (10 min)

1. Login to **Super Admin**
2. Click **Onboard restaurant**
3. Fill in:
   - Restaurant name: `QA Test Kitchen` (or unique name)
   - First outlet: `Main Outlet`
   - Plan: `enterprise`
   - Owner name, email, password (save in password manager)
4. Copy the success message — it has the Admin URL and owner email
5. Write **org slug** and **outlet slug** in your credential worksheet

Step 3 — Owner setup (10 min)

1. Login to **Admin** with owner credentials
2. Go to **Setup** (`/onboarding`) — choose business type **Restaurant**
3. Complete the wizard (business info, GSTIN test value `27AAAAA0000A1Z5`, etc.)
4. **Menu** (`/menu`) — create 2 categories and at least 4 items
5. **Staff** (`/staff`) — add **Waiter** and **Cashier**, assign main outlet
6. **Tables** (`/tables`) — create at least 2 tables (T1, T2)
7. Share waiter login with your employee (securely — not in git)

Step 4 — First real test (5 min)

Employee: Login to Waiter → select outlet → open a table → add items → confirm order

You: Check **Admin** → **Orders** — the order should appear; optionally confirm KOT on https://kds.cullinos.com

If **no tables** show in Waiter: create them in Admin → **Tables**, or via Swagger (`POST /api/v1/tables`) if the UI fails.

What to test next (4-day plan)

Day	Focus	Document
1	Preflight + onboard + menu + tables + staff	This guide + Phase 1 in Manual Test Plan
2	Waiter, POS, KDS, guest checkout, CDS	Phase 2 + mark Test Run Sheet
3	Admin modules (incl. new pages), Management, Super Admin	Phases 3–5
4	Marketing site + API (Swagger)	Phases 6–7

Full details: MANUAL_TEST_PLAN.md
Track pass/fail: TEST_RUN_SHEET.md (**232** test cases)
Click-paths: COMPLETE_FEATURE_TEST_GUIDE.md
Log bugs: BUG_LOG.md What's new: WHATS_NEW.md

App cheat sheet — what each page does

Admin (owner) — https://admin.cullinos.com

Menu	Path	What to check
Dashboard	`/`	Revenue, order counts after you place test orders
Portal POS	`/pos`	Hold / resume / checkout (needs POS permission)
Menu	`/menu`	Add/edit categories, items, combos, schedules, outlet prices
Orders	`/orders`	All orders from Waiter, Guest, POS
Tables	`/tables`	Create/edit tables — real UI
Reservations	`/reservations`	Bookings + public book link
Inventory	`/inventory`	List / add / adjust stock — real UI
Customers	`/customers`	Create / search customers
Loyalty	`/loyalty`	Settings, tiers, rewards
Coupons	`/coupons`	Create / deactivate coupons
Recipes	`/recipes`	Create and list recipes
Purchasing / Suppliers	`/purchasing` , `/suppliers`	Draft PO, GRN, supplier directory
Central kitchen	`/central-kitchen`	Often N/A until multi-outlet
Delivery	`/delivery`	Zones / delivery orders
Aggregators	`/aggregators`	Swiggy / Zomato connect
Payments	`/payments`	Razorpay / Cashfree keys
App listing	`/marketplace`	Cullinos App listing
Guest banners	`/guest-banners`	App banners
SMS campaigns	`/sms-campaigns`	MSG91 SMS (needs wallet)
Billing	`/billing`	Billing / subscription page
Events	`/events`	Schedule events; select outlet, fill form
Production	`/production`	Schedule batches, mark complete
Banquets	`/banquets`	Often **N/A** on restaurant tenant
Brands	`/brands`	Often **N/A** on restaurant tenant
Guests / Rooms	`/hospitality/*`	Often **N/A** on restaurant tenant
Displays	`/displays`	KDS / CDS / Pickup / Promo URLs (legacy `/kds` `/cds` redirect here)
Digital Ordering	`/kiosk`	Kiosk / storefront launcher
Staff	`/staff`	Create waiter/cashier/manager logins
Reports	`/reports`	Revenue, top items, Export

Menu	Path	What to check
Settings	`/settings`	Form: restaurant details, order types, locations, tax, devices
Setup	`/onboarding`	Initial wizard (seeds sample categories on Complete setup)
Forgot / Change password	`/forgot-password`, `/change-password`	Auth recovery

Note: There is no Admin "Demo data" button. Sample menu categories come from **Complete setup**. Promo Email is on **Super Admin** only.

Super Admin — https://platform.cullinos.com

Menu	Path	What to check
Tenants	`/` or `/tenants`	List restaurants, onboard, suspend, impersonate
Plans	`/plans`	Plan catalog
Subscriptions	`/subscriptions`	Change plan per tenant
Promo Email	`/promo-email`	Platform promo campaigns (not in Admin)
Cullinos App / Guest Ops	`/guest-ops/*`	Marketplace, discover, banners, push, offers, reviews, users, analytics, runtime
Settings	`/settings`	Platform settings
System Health	`/health`	Platform metrics
Marketing	`/marketing/*`	Edit website content (careful on production)
Testimonials	`/marketing/testimonials`	Testimonials editor
Design Lab	`/marketing/design-lab`	Design lab page

Management — https://manage.cullinos.com

Menu	Path	What to check
Overview	`/`	Multi-outlet KPIs
Reports	`/reports`	Network reports
Comparison	`/comparison`	Needs 2+ outlets
Stock Transfer	`/stock-transfer`	Needs 2+ outlets
Franchise	`/franchise`	Franchise list

Cullinos Waiter — Android app

1. Login → pick outlet
2. Tap a table
3. Add menu items → confirm → order goes to kitchen

Cullinos App (Guest) — deep link / Android

Use `https://guest.cullinos.com/o/{orgSlug}/{outletSlug}` (or the Android app).

1. Browse menu (guest OK)
2. Optional: phone OTP / Google login
3. Cart → Checkout (auth required for checkout)
4. Pay later or UPI → order placed
5. Also test: Home, Explore, Scan, Offers, Profile, Kiosk mode

Full steps: COMPLETE_FEATURE_TEST_GUIDE.md chapter 10.

CRM & Loyalty — how to test

The Admin **Customers** page (`/customers`) is a read-only display of loyalty tiers and active coupons. Also open **Loyalty** (`/loyalty`). There is no Admin UI to add stamps — stamps are added via API call.

What is pre-seeded (demo org)

Item	Value
Loyalty tier	Stamp Card — 10 stamps = 100 loyalty points = free drink
Coupon	`WELCOME10` — 10% off, minimum order ₹200

Step-by-step loyalty test

1. **Place an order** as a guest customer (Customer app → checkout) so a customer record is created.
2. **Find the customer ID**: open Swagger (`/docs`) → `GET /api/v1/customers` → Authorise with owner Bearer token → Execute. Copy the `id` from the response.
3. **Add stamps** via Swagger → `POST /api/v1/loyalty/customers/{customerId}/stamp` — execute it 10 times.
4. After 10 stamps the system automatically adds 100 loyalty points and resets the stamp count.
5. **Verify on Admin Customers page**: tier list and coupon list load without error → **Pass**.

Step-by-step coupon test

1. Add items to cart in the Customer app (total ≥ ₹200).
2. At checkout, enter coupon code **`WELCOME10`** in the coupon field.
3. Verify 10% discount is applied to the total.
4. Complete the order.

Test case results

Check	Expected	Result
Admin Customers page loads	Tier list + coupon list visible	Pass/Fail
Coupon `WELCOME10` applies at checkout	10% discount on orders ≥ ₹200	Pass/Fail
Stamp endpoint reachable via Swagger	HTTP 200, stamp count increments	Pass/Fail

---

Local setup (optional fallback)

Only needed if production POS/KDS URLs are unavailable.

Prerequisites: Node.js, repo cloned, `npm install`

```
# Terminal 1 — POS (cashier)
cd a:\cullinos
set VITE_API_URL=https://api.cullinos.com/api/v1
set VITE_WS_URL=https://api.cullinos.com
npm run dev --workspace=@cullinos/pos

# Terminal 2 — KDS (kitchen)
set VITE_API_URL=https://api.cullinos.com/api/v1
set VITE_WS_URL=https://api.cullinos.com
npm run dev --workspace=@cullinos/kds
```

Then open:

- POS: http://localhost:5173 (login as **Cashier** — create in Admin → Staff)
- KDS: http://localhost:5174?outletId=**YOUR_OUTLET_ID**

Get `outletId` from Admin Settings, Swagger, or your credential worksheet.

Common issues (not bugs)

Problem	What to do
No tables in Waiter	Create in Admin → Tables (or Swagger if UI fails)
Banquets / Brands / Guests / Rooms missing	Expected on restaurant tenant — mark N/A
Management comparison empty	Add a second outlet via API, or mark test Blocked
Pay now fails on Customer	Razorpay may not be configured — use Pay later
Login “Failed to fetch”	Frontends must point to `api.cullinos.com` — tell your lead if prod is misconfigured

---

Files you will use daily

File	Use
This guide	URLs, credentials worksheet, first-day steps
MANUAL_TEST_PLAN.md	Full workflows and expected results
TEST_RUN_SHEET.md	Mark Pass / Fail / Blocked / N/A per test
BUG_LOG.md	Record every failure with steps to reproduce
CREDENTIALS_LOG.md	Detailed credential template for each run

Per run: copy templates to `docs/qa/runs/RUN-YYYYMMDD/` (private, not committed).

Quick reference card (print or pin)

```
PRODUCTION PORTALS
──────────────────────────────────────────────────
Super Admin   https://platform.cullinos.com
Admin         https://admin.cullinos.com
Portal POS    https://admin.cullinos.com/pos
Management    https://manage.cullinos.com
Waiter        Cullinos Waiter Android app
POS           https://pos.cullinos.com
KDS           https://kds.cullinos.com
Customer      https://guest.cullinos.com/{orgSlug}/{outletSlug}
Marketing     https://cullinos.com
Swagger       https://api.cullinos.com/docs
Health        https://api.cullinos.com/api/v1/health

TEST ORDER
──────────────────────────────────────────────────
1. Super Admin → Onboard (enterprise plan)
2. Admin → Menu + Tables + Staff (Waiter + Cashier)
3. Waiter → Table order → KDS verifies
4. POS / Portal POS → checkout
5. Customer → Checkout incognito (verify)
6. TEST_RUN_SHEET → mark all 232 cases

NEVER commit real passwords to git.
```

Need help?

1. Read MANUAL_TEST_PLAN.md for the phase you're on
2. Check ARCHITECTURE.md for how apps connect
3. Ask your team lead for Super Admin access and tenant slugs
4. Log blockers in BUG_LOG.md with screenshots